

When we loop through the dictionary, the value associated with each person would be a list of languages rather than a single language. Inside the dictionary's for loop, we use another for loop to run through the list of languages associated with each person:

```
favorite_languages.py favorite_languages = {
    'jen': ['python', 'rust'],
    'sarah': ['c'],
    'edward': ['rust', 'go'],
    'phil': ['python', 'haskell'],
}
```

- ❶ for name, languages in favorite_languages.items():
 print(f"\n{name.title()}'s favorite languages are:")
- ❷ for language in languages:
 print(f"\t{language.title()}")

The value associated with each name in `favorite_languages` is now a list. Note that some people have one favorite language and others have multiple favorites. When we loop through the dictionary ❶, we use the variable name `languages` to hold each value from the dictionary, because we know that each value will be a list. Inside the main dictionary loop, we use another for loop ❷ to run through each person's list of favorite languages. Now each person can list as many favorite languages as they like:

Jen's favorite languages are:

Python
Rust

Sarah's favorite languages are:

C

Edward's favorite languages are:

Rust
Go

Phil's favorite languages are:

Python
Haskell

To refine this program even further, you could include an if statement at the beginning of the dictionary's for loop to see whether each person has more than one favorite language by examining the value of `len(languages)`. If a person has more than one favorite, the output would stay the same. If the person has only one favorite language, you could change the wording to reflect that. For example, you could say, "Sarah's favorite language is C."

NOTE

You should not nest lists and dictionaries too deeply. If you're nesting items much deeper than what you see in the preceding examples, or if you're working with someone else's code with significant levels of nesting, there's most likely a simpler way to solve the problem.